Python с нуля: от первой строки до проекта

Наглядный учебник для начинающих



BOOKICH AI

Python с нуля: от первой строки до проекта

Пошаговый курс, который проведёт вас от установки Python до собственного приложения. Каждая глава объясняет одну идею простыми словами, показывает её на схемах и закрепляет практикой.

#python #программирование #с нуля #практика #схемы

СОДЕРЖАНИЕ

1 Знакомство с Python

- Что такое Python и где он применяется
- Установка и первый запуск
- Ваша первая программа

2 Переменные и типы данных

- Переменные и присваивание
- Числа и операции
- Строки и f-строки

3 Условия и циклы

- Ветвления if / elif / else
- Цикл for и range
- Цикл while и break

4 Коллекции

- Списки и кортежи
- Словари
- Множества и генераторы

5 Функции

- Объявление функций
- Аргументы по умолчанию и *args
- Область видимости

6 Работа с файлами

- Открытие и чтение файлов
- Запись и CSV
- Исключения

7 Модули и пакеты

Импорт модулей

pip и виртуальные окружения

Полезные модули

8 Первый проект

Проектируем приложение

Пишем код

Что изучать дальше

ГЛАВА 1

Знакомство с Python

Почему Python так популярен и как подготовить рабочее место.

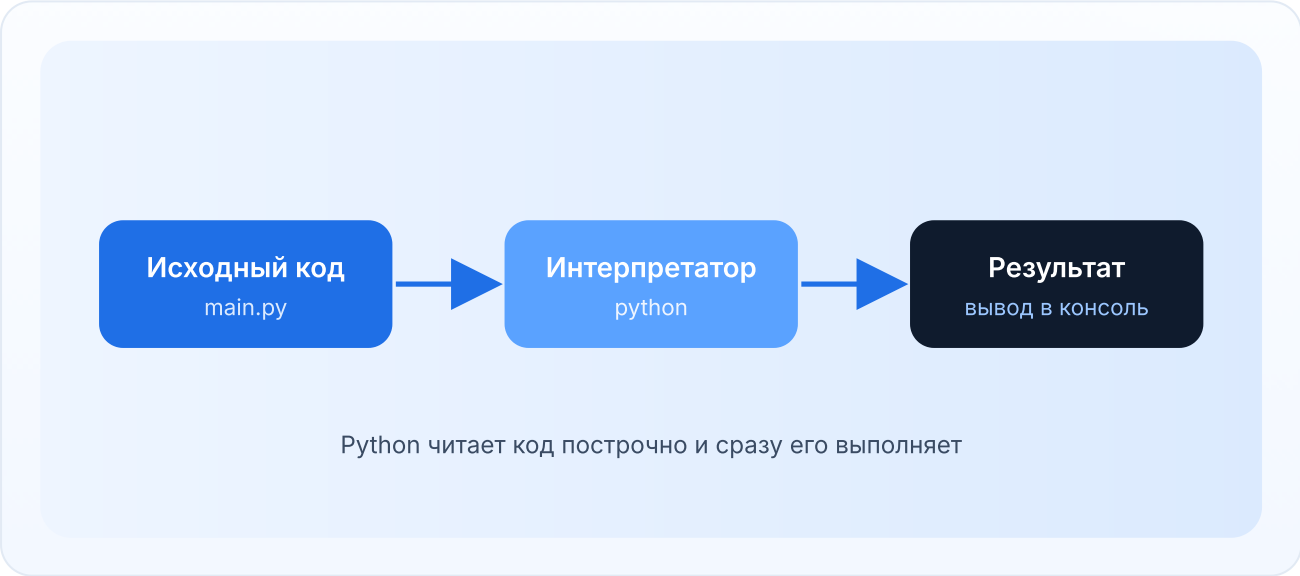
Основа Рекомендуется

Что такое Python и где он применяется

Главная идея

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Установка и первый запуск

Главная идея

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

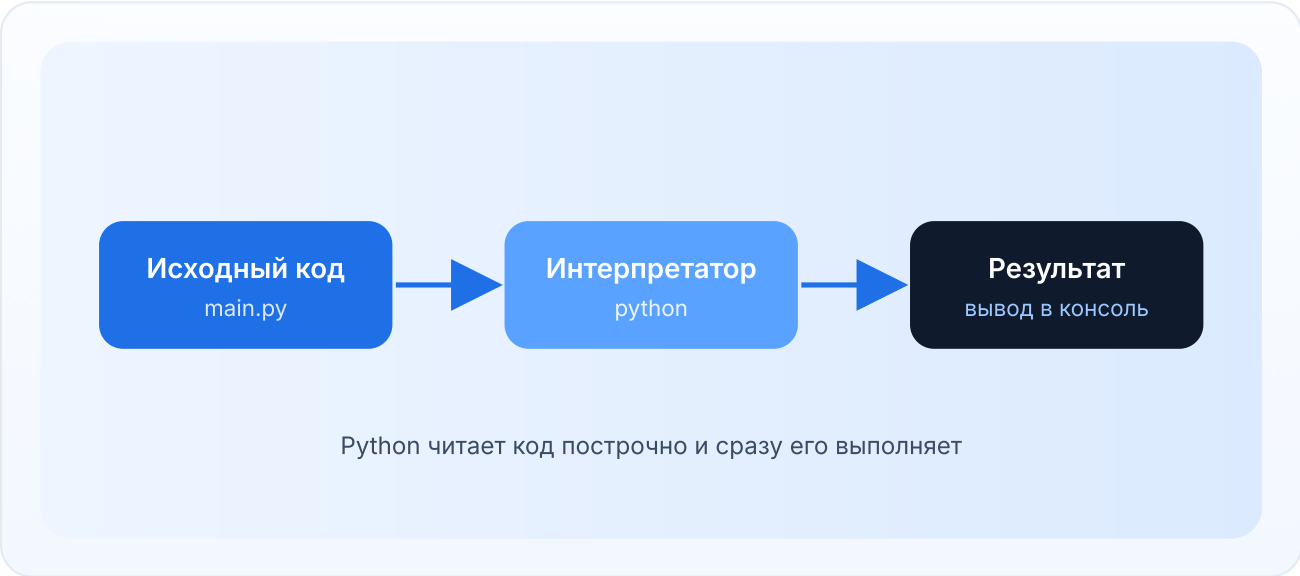
Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Ваша первая программа

Главная идея

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

ГЛАВА 2

Переменные и типы данных

Числа, строки, логика и то, как Python хранит значения.

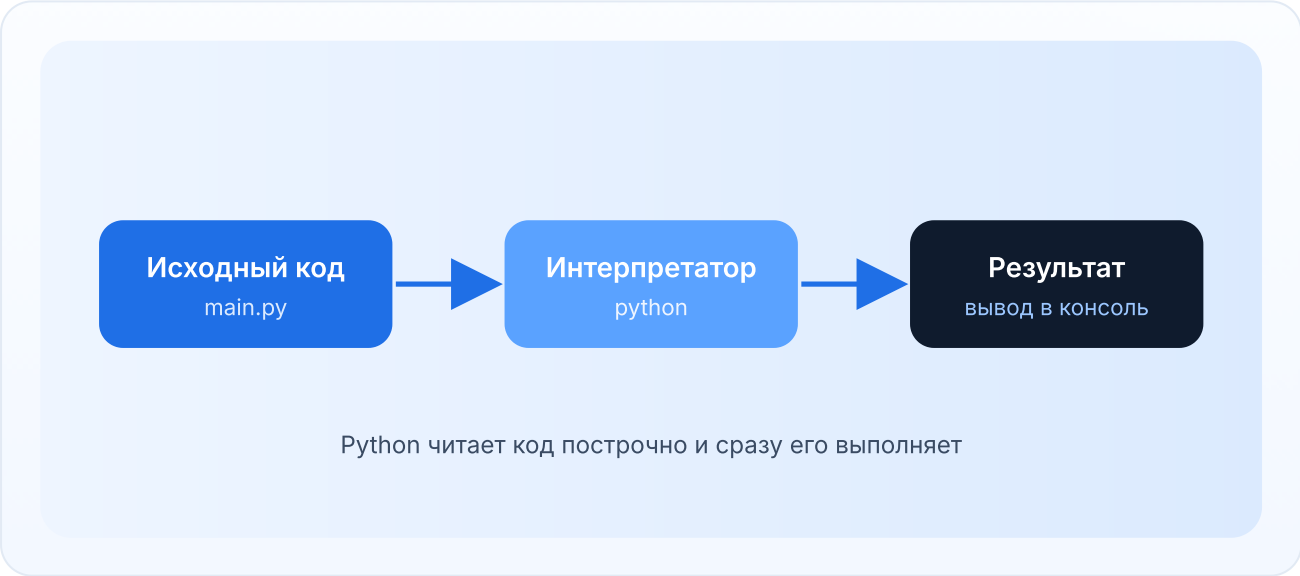
Основа

Переменные и присваивание

Главная идея

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Числа и операции

Главная идея

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

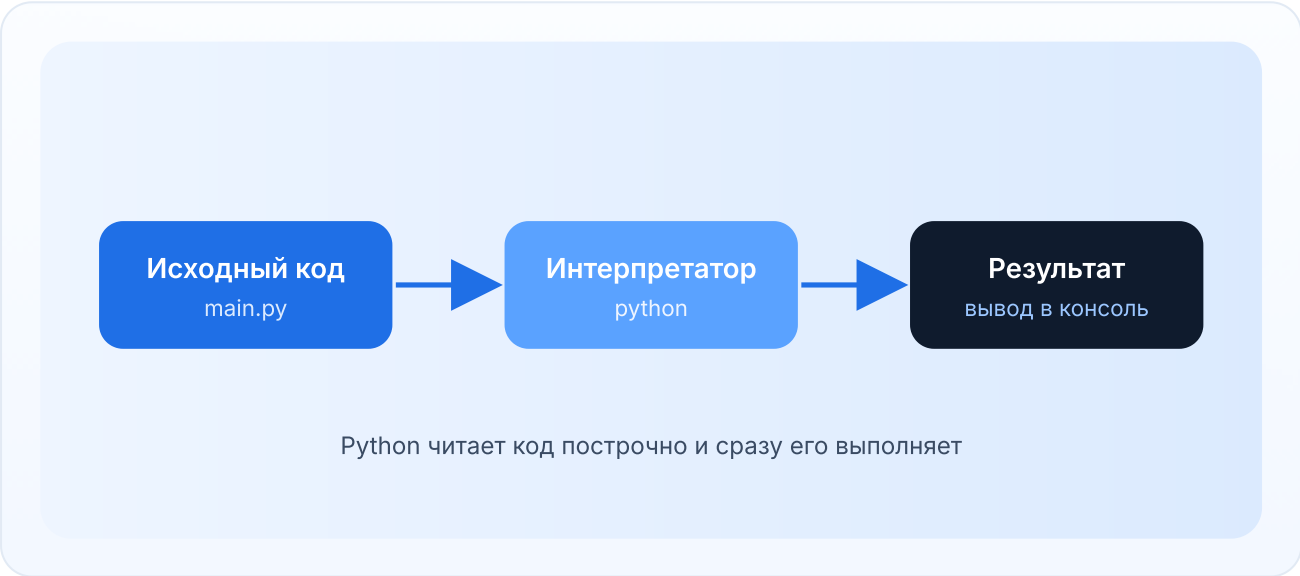
Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Строки и f-строки

Главная идея

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

ГЛАВА 3

Условия и циклы

Управляем ходом программы: if, for, while.

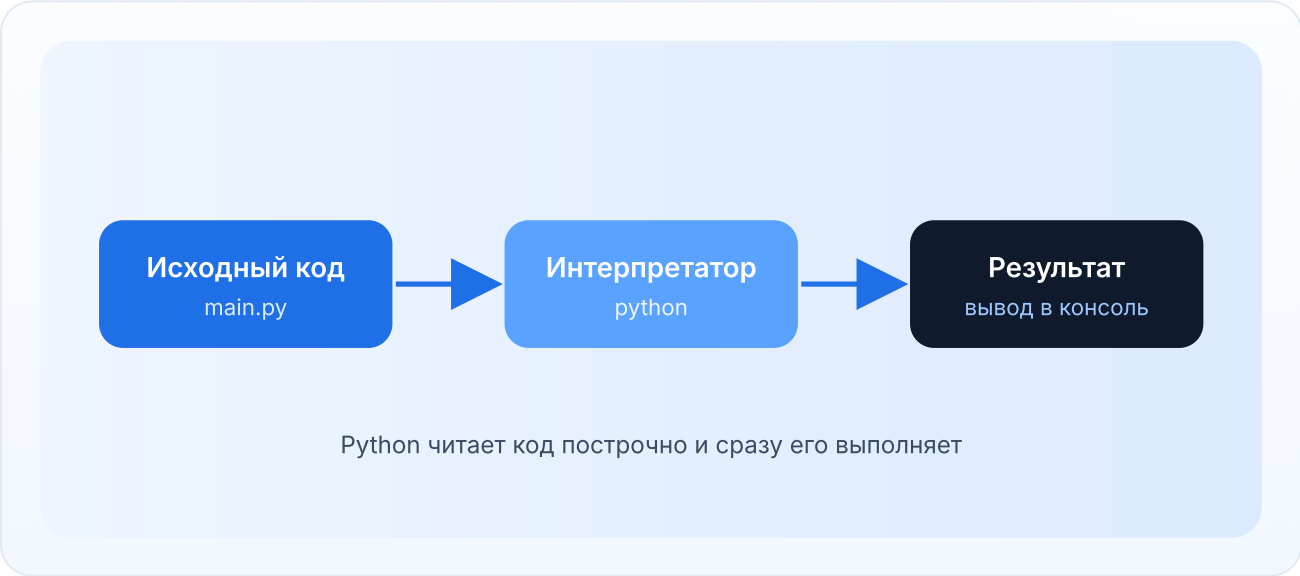
Рекомендуется Практика

Ветвления if / elif / else

Главная идея

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Цикл for и range

Главная идея

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

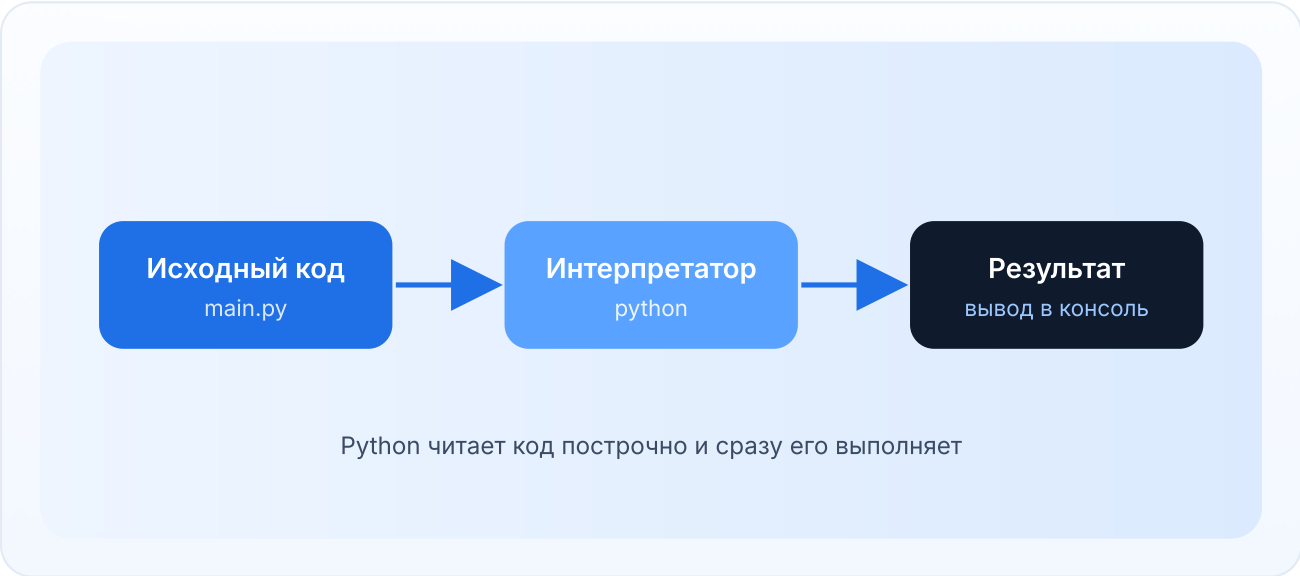
Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Цикл while и break

Главная идея

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

ГЛАВА 4

Коллекции

Списки, кортежи, словари и множества на реальных задачах.

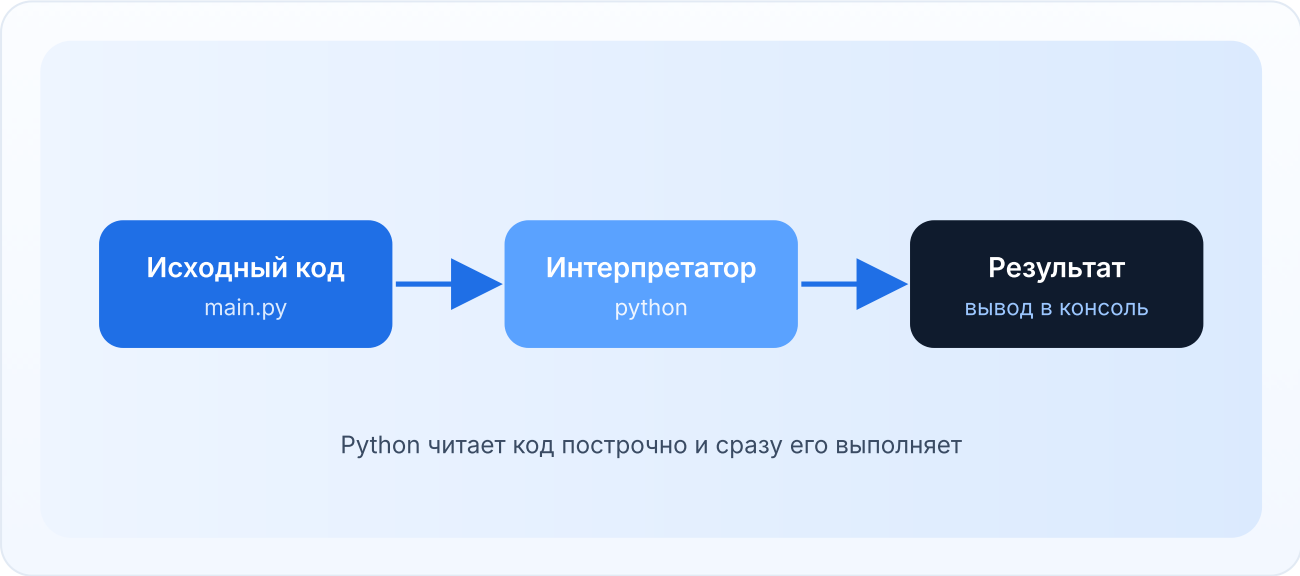
Интересно

Списки и кортежи

Главная идея

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Словари

Главная идея

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

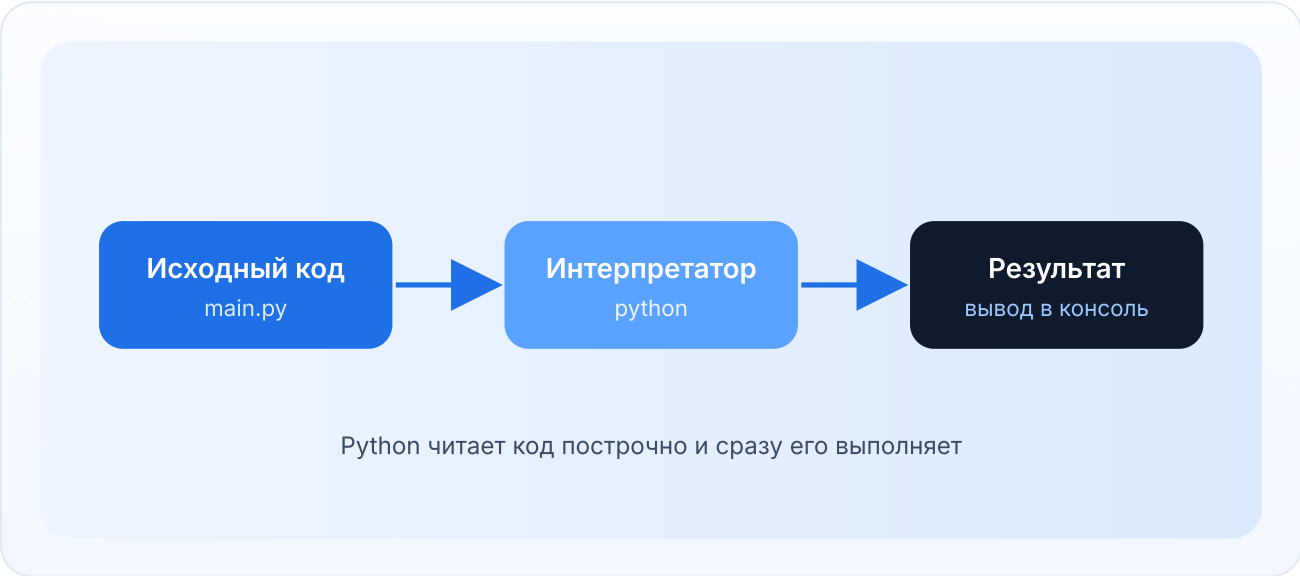
Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Множества и генераторы

Главная идея

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

ГЛАВА 5

Функции

Как разбивать код на понятные переиспользуемые блоки.

Рекомендуется

Объявление функций

Главная идея

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.



Python читает код построчно и сразу его выполняет

Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5`?

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Аргументы по умолчанию и *args

Главная идея

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600`: второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

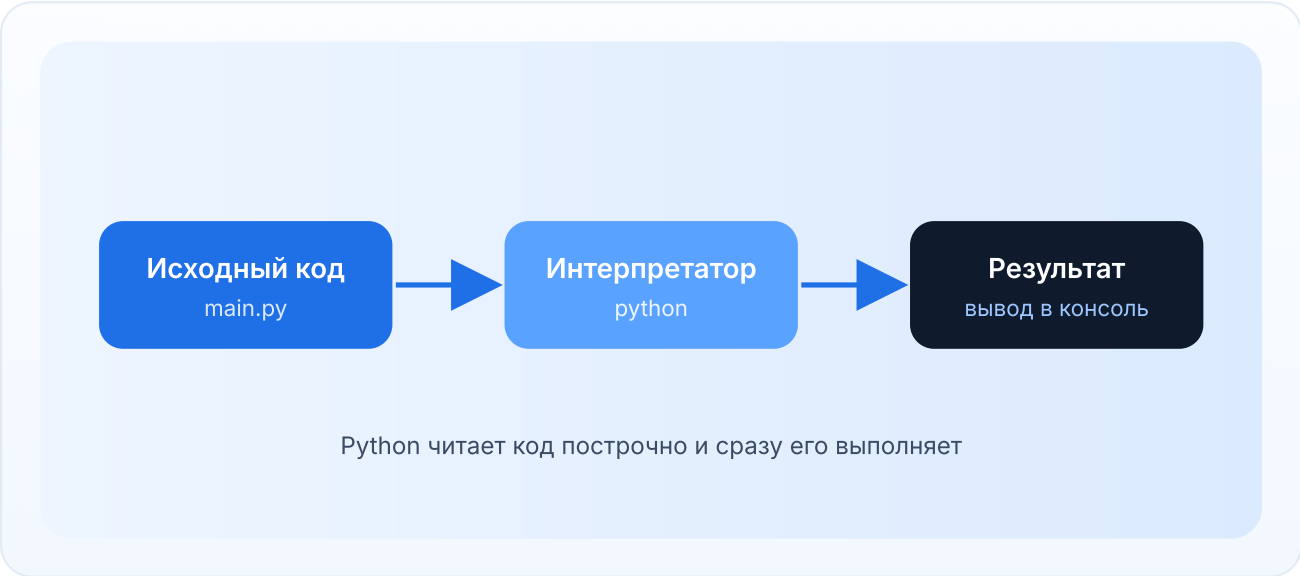
Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Область видимости

Главная идея

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

ГЛАВА 6

Работа с файлами

Читаем и записываем данные, обрабатываем ошибки.

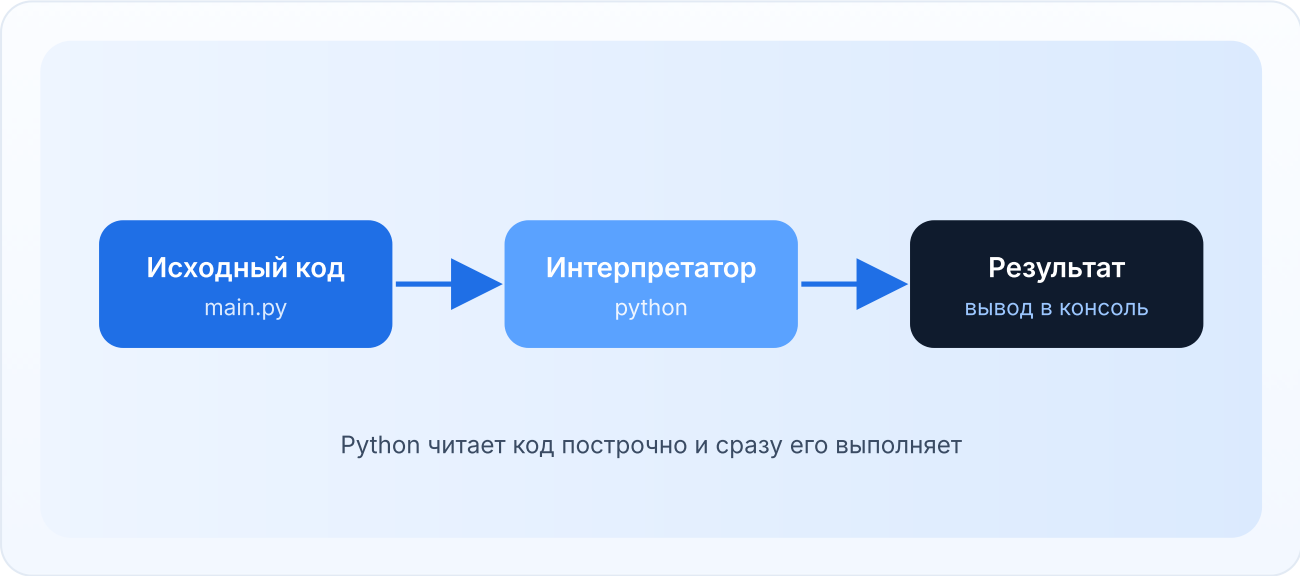
Практика

Открытие и чтение файлов

Главная идея

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Запись и CSV

Главная идея

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

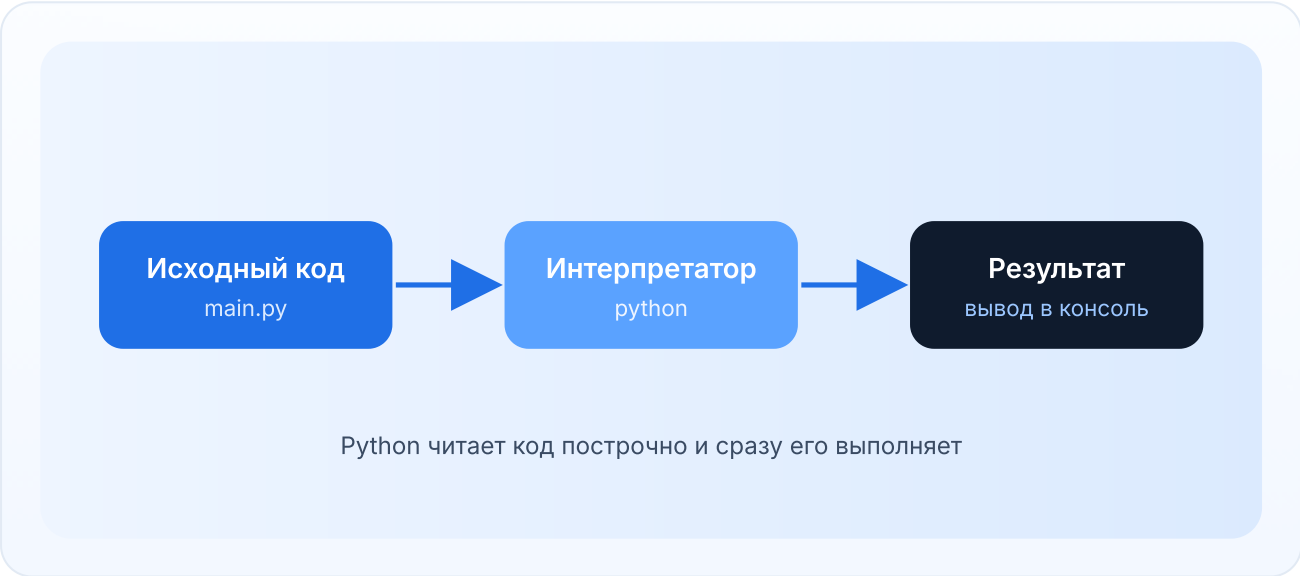
Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Исключения

Главная идея

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

ГЛАВА 7

Модули и пакеты

Стандартная библиотека и установка сторонних пакетов.

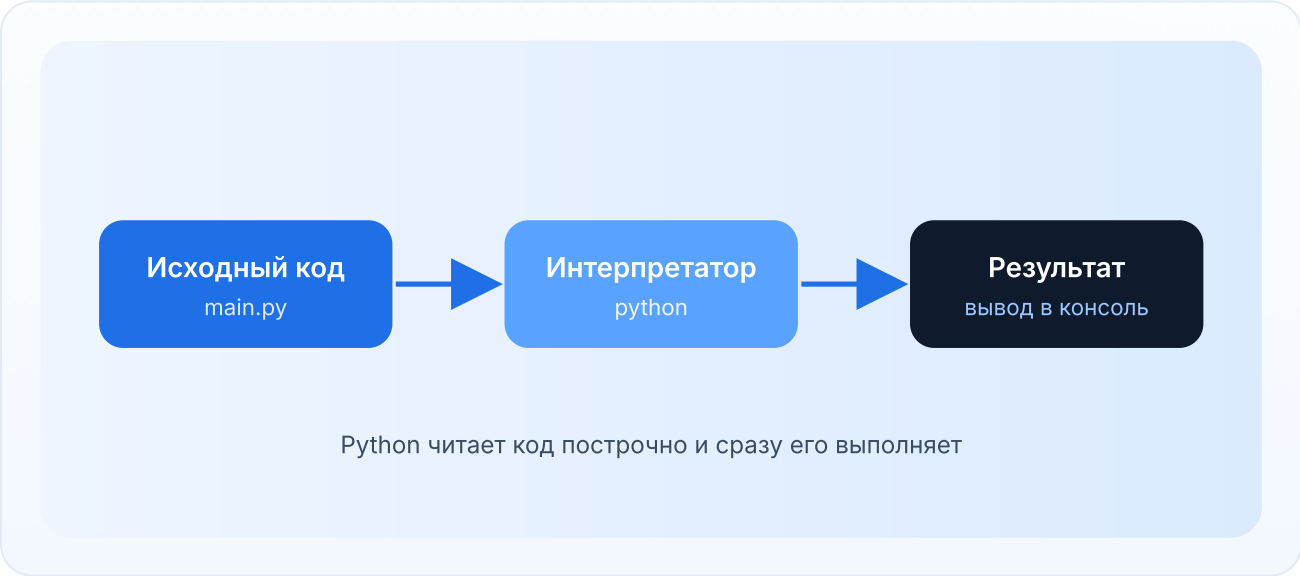
Углублённо

Импорт модулей

Главная идея

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

pip и виртуальные окружения

Главная идея

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5`?

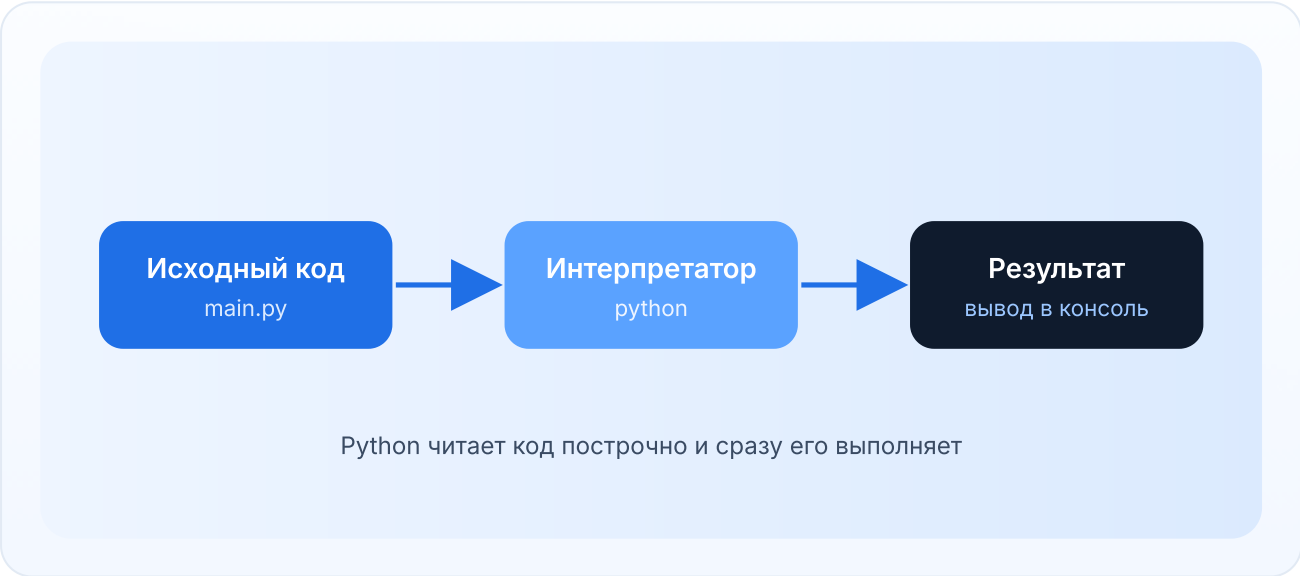
Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Полезные модули

Главная идея

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

ГЛАВА 8

Первый проект

Собираем всё вместе: консольный менеджер задач.

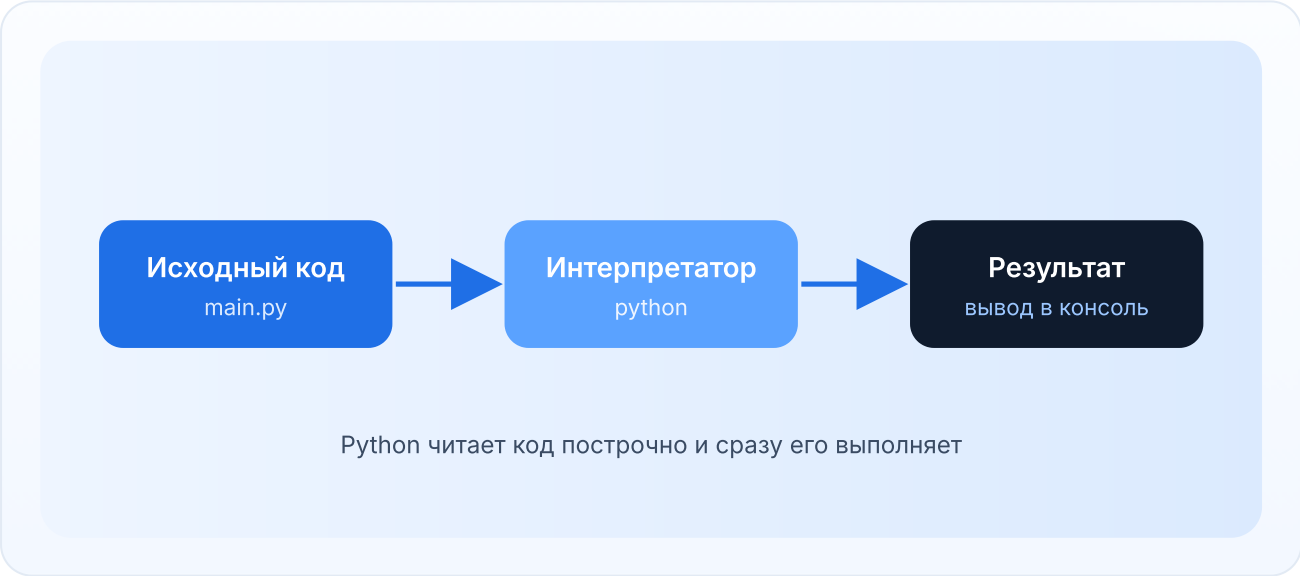
Практика Интересно

Проектируем приложение

Главная идея

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Пишем код

Главная идея

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

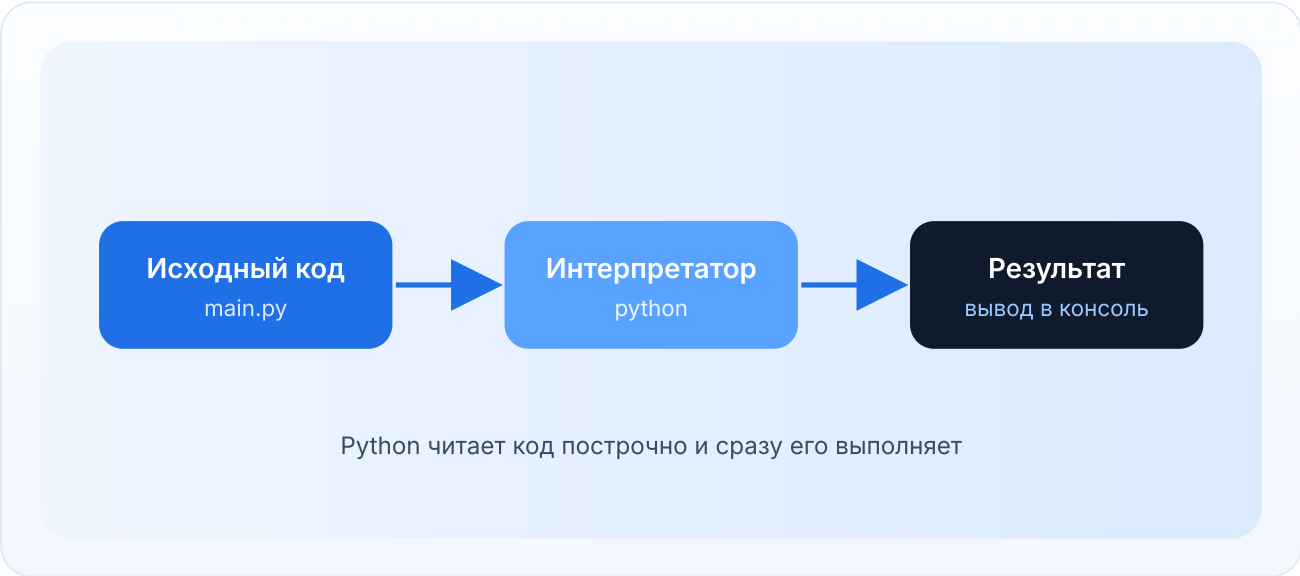
Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Что изучать дальше

Главная идея

Отступы в Python - не просто оформление, а часть синтаксиса. Блок кода внутри условия или цикла выделяется четырьмя пробелами. Если отступ сбит, программа не запустится, и это защищает от целого класса трудноуловимых ошибок.

Python читает программу сверху вниз, строка за строкой. Каждая инструкция выполняется сразу, поэтому результат виден мгновенно: напишите команду, запустите файл и посмотрите, что получилось. Такой цикл «написал - проверил» делает обучение быстрым и наглядным.



Совет: запускайте код после каждого небольшого изменения - так ошибки находятся сразу.

Пример

```
name = input("Как вас зовут? ")
age = int(input("Сколько вам лет? "))
print(f"Привет, {name}! Через 10 лет вам будет {age + 10}.")
```

Переменная в Python - это имя, которое указывает на значение в памяти. Вам не нужно заранее объявлять её тип: интерпретатор сам поймёт, что `age = 25` - это целое число, а `name = "Анна"` - строка. Это удобно, но требует внимательности к тому, что хранится в каждой переменной.

Тип	Пример	Назначение
int	`42`	Целые числа
float	`3.14`	Дробные числа
str	`"Привет"`	Текст
bool	`True`	Логические значения

Проверьте себя: чем отличается `"5" + "5"` от `5 + 5` ?

Хорошая привычка - давать переменным говорящие имена. Сравните `x = 3600` и `seconds_in_hour = 3600` : второй вариант объясняет сам себя, и через месяц вы без труда поймёте собственный код.